Analyze output to prioritize & prepare attacks

This is the **chess-player** phase — the analytical work between finishing recon and firing the first exploit. You take raw scan output and turn it into a ranked, scoped, documented attack plan. Two halves: **prioritize the targets** (Part 1), then **select the capabilities** to hit them (Part 2). The exam tests judgment, so the wording is scenario-based: which first, which tool, which is safe to touch.

---

§00 the workflow that ties it together

Every concept in this guide is a stage in one pipeline. Hold this spine in your head — most 4.1 questions are really asking "which stage am I in, and what does that stage optimize for?"

1. **Identify high-value assets**
   Find the DCs, databases, payment systems; cross-reference the SoW objective.
2. **Apply the metrics**
   Score with CVE + CVSS + CWE + EPSS. High CVSS + high EPSS = attack first.
3. **Assess target characteristics**
   EOL? Default configs? Running services? Weak crypto? Defensive gaps?
4. **Select capabilities**
   Match tools to targets, pick and analyze exploit code, map dependencies.
5. **Document the plan**
   Attack paths, low-level diagrams, storyboards; label sensitive systems; verify scope.
6. **Execute**
   Everything downstream — Domains 4.2 through 4.10.

PART 1 Target prioritization

Ranking targets by value, vulnerability, and exploitability so limited time hits what matters.

1.1 high-value asset identification

High-value assets are systems whose compromise causes the greatest business impact. Attacking a low-value box wastes engagement time — so this is step one of prioritization.

`what makes an asset "high value"`

- **Authentication servers** — AD Domain Controllers, LDAP, RADIUS. One compromised DC can cascade into full enterprise compromise.
- **Databases with sensitive data** — PII, financial, IP, healthcare. Highest business impact if breached.
- **Financial / ERP systems** — SAP, Oracle Financials, payment infrastructure. Direct financial risk.
- **Customer-facing web apps** — most exposed; a breach can affect thousands of users.

- **Certificate Authorities / PKI** — forge trusted certs, enabling org-wide MITM.
- **Backup & recovery systems** — often overlooked; enable ransomware-style destruction.
- **Network infrastructure** — firewalls, routers, switches. Visibility into and control of all traffic.
- **Admin interfaces & jump servers** — bastion hosts, PAWs, management consoles give privileged reach.

how to identify them on an engagement

- Read the **Statement of Work** — the client usually names what they most want protected.
- Cross-reference **OSINT** — job postings and tech disclosures reveal critical systems.
- Analyze **scan results** — many open ports, or DB/auth services, stand out.
- Read **DNS naming conventions** — `dc01`, `sqlprod`, `payroll`, `backup` are giveaways.
- Review **BloodHound** output — maps AD privilege paths and the most critical accounts/systems.

> **EXAM TELL** When asked **which system to target first** from a host list, Domain Controllers and database servers are almost always the answer. "High value" is decided by **business impact, not technical exposure** — the box with the most open ports isn't automatically the pick.

1.2 descriptors & metrics

Objective scoring systems let you rank vulnerabilities by measurable data instead of intuition. Four acronyms — and the exam cares most about how they differ and combine.

1.2.1 CVSS — severity of the vulnerability

The Common Vulnerability Scoring System rates severity. Current version is **CVSS 4.0** (you'll still see 3.1 in the wild). The **base score** captures intrinsic traits that don't change with environment or time, built from two metric groups:

EXPLOITABILITY METRIC	WHAT IT MEASURES	VALUES
Attack Vector (AV)	How the attacker reaches the component	Network (N), Adjacent (A), Local (L), Physical (P)
Attack Complexity (AC)	Conditions beyond the attacker's control	Low (L), High (H)
Attack Requirements (AT)	Prerequisites for exploitation (new in 4.0)	None (N), Present (P)
Privileges Required (PR)	Access needed before exploitation	None (N), Low (L), High (H)
User Interaction (UI)	Whether a user must act	None (N), Passive (P), Active (A)

IMPACT METRIC	WHAT IT MEASURES	VALUES
Confidentiality (VC/SC)	Impact on data confidentiality	None / Low / High
Integrity (VI/SI)	Impact on data integrity	None / Low / High
Availability (VA/SA)	Impact on system availability	None / Low / High

SCORE RANGE	SEVERITY
0.0	None
0.1 — 3.9	Low
4.0 — 6.9	Medium
7.0 — 8.9	**High**
9.0 — 10.0	**Critical**

Always read the whole vector, not just the number — a 9.8 that needs physical access is nothing like a 9.8 that's remotely exploitable:

```
CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

AV:NAttack Vector: Network — remotely exploitable
AC:LAttack Complexity: Low — no special conditions
AT:NAttack Requirements: None
PR:NPrivileges Required: None — no authentication
UI:NUser Interaction: None — no victim action
VC/VI/VA:HFull confidentiality, integrity, availability impact
```

EXAM TELL Memorize the ranges (Critical 9.0–10.0, High 7.0–8.9, Medium 4.0–6.9, Low 0.1–3.9), know **CVSS 4.0 is current**, and burn in the perfect-storm signature: **AV:N + PR:N + UI:N** = remotely exploitable, unauthenticated, no user action = highest exploitability. The base score's limitation — it ignores whether an exploit exists in the wild — is exactly what EPSS fixes.

1.2.2 CVE — the specific named vulnerability

Common Vulnerabilities and Exposures is a naming system for publicly known flaws, maintained by **MITRE**. Format is `CVE-YEAR-NUMBER`. It's the universal reference tools and vendors share.

FAMOUS CVE	NAME
CVE-2021-44228	Log4Shell — Apache Log4j RCE
CVE-2017-0144	EternalBlue — SMB (WannaCry)
CVE-2021-34527	PrintNightmare — Windows Print Spooler

A CVE entry carries the ID, description, references (advisories, patches, exploit code), CVSS score, the **CWE** it stems from, and **CPE** (affected products). Testers use it to confirm a scanner finding in the NVD (nvd.nist.gov), find exploit code (Exploit-DB, Metasploit, GitHub), understand the vector, check for patches, and prioritize alongside EPSS.

EXAM TELL Know CVE is **MITRE**-maintained, the `CVE-YEAR-NUMBER` format, and — the distinction they love — **CVE = a specific vulnerability in a specific product; CWE = the weakness category that caused it**. Log4Shell (CVE-2021-44228) is caused by CWE-502 (deserialization) and CWE-20 (input validation).

1.2.3 CWE — the weakness category

Common Weakness Enumeration (also MITRE) classifies the type of flaw behind a vulnerability. It helps you predict siblings (a CWE-89 SQLi in one field suggests others), understand root cause, and prioritize by weakness class.

CWE	NAME	DESCRIPTION
CWE-79	Cross-Site Scripting	Improper neutralization of input in web pages
CWE-89	SQL Injection	Improper neutralization of SQL special elements
CWE-20	Improper Input Validation	Failure to validate input properly
CWE-125	Out-of-bounds Read	Reading beyond buffer boundaries
CWE-787	Out-of-bounds Write	Writing beyond buffer boundaries
CWE-502	Deserialization of Untrusted Data	Processing untrusted serialized data
CWE-22	Path Traversal	Improper limitation of a pathname
CWE-306	Missing Authentication	Critical function without authentication
CWE-862	Missing Authorization	Insufficient authorization checks
CWE-287	Improper Authentication	Authentication that can be bypassed

EXAM TELL Recognize the headline numbers on sight: **CWE-79 (XSS), CWE-89 (SQL injection), CWE-22 (path traversal)**. CWE is what lets you predict where **similar** vulnerabilities hide in the same app — a CWE-89 in a DB-connected web app is instant high priority (sqlmap automates it; impact is severe).

1.2.4 EPSS — probability of exploitation

The Exploit Prediction Scoring System predicts the **probability a vuln will be exploited in the wild within 30 days**, scored **0.0–1.0** and updated daily from threat intel, exploit-code availability, forum/dark-web chatter, and historical patterns. CVSS measures severity; EPSS measures likelihood.

The problem it solves: only about **4–7% of published CVEs are ever exploited**, so treating every Critical CVSS equally burns resources on theoretical risk.

VULN	CVSS	EPSS	PRIORITY
CVE-A	9.8 Critical	0.3%	Lower — theoretical risk
CVE-B	6.5 Medium	84.7%	Higher — actively exploited
CVE-C	9.8 Critical	92.1%	**Highest — severe AND likely**

The ideal method combines both scores:

High EPSS

Low EPSS

ATTACK FIRST
Severe and actively exploited — immediate priority.

IMPORTANT
Severe but not yet exploited — less urgent.

SURPRISING PRIORITY
Actively exploited despite lower severity — don't skip.

LOWEST
Neither severe nor exploited.

EXAM TELL The classic trap: a **Medium-CVSS + high-EPSS** vuln outranks a **Critical-CVSS + low-EPSS** one, because it's actually being exploited. Remember: high EPSS = exploit code likely exists and is in use. EPSS = likelihood, CVSS = severity — combine them.

1.3 end-of-life (EOL) software

EOL software is the most reliably exploitable condition you'll meet. Once a vendor stops shipping patches, vulnerabilities remain **permanently** unpatched and exploit code accumulates.

SOFTWARE	EOL	WHY IT MATTERS
Windows XP	Apr 2014	EternalBlue, countless unpatched CVEs
Windows 7	Jan 2020	Still common in enterprises
Windows Server 2008	Jan 2020	Widely deployed, many unpatched flaws
Internet Explorer	Jun 2022	Numerous unpatched browser bugs
PHP 5.x	Dec 2018	Widely deployed on web servers
Apache 2.2	Dec 2017	Common on legacy web servers
OpenSSL 1.0.x	Dec 2019	Heartbleed and other critical flaws

EOL boxes are high priority because the vuln will never be patched, public exploit code is plentiful, EPSS scores tend to be high, and organizations running EOL usually have weak hygiene overall. You find them through version detection: `nmap -O` (OS fingerprint), `nmap -sV` (service versions), banner grabbing, scanner output, and HTTP `Server:` headers.

EXAM TELL **EOL = no vendor patches = permanent vulnerability**, and it should be prioritized highly. EOL vulns often carry **high EPSS** because exploits are actively used. The way you spot EOL is **version detection** — `nmap -sV` and banner grabbing.

1.4 default configurations

Out-of-box settings — default credentials, unneeded services, insecure defaults — are low-effort, high-reward. Trivially exploitable, widely documented, and surprisingly common.

DEVICE / SYSTEM	DEFAULT USER	DEFAULT PASSWORD
Cisco IOS	cisco	cisco
Linksys routers	admin	admin
NETGEAR	admin	password
VMware ESXi	root	(blank)
MySQL	root	(blank)
Tomcat Manager	admin	admin
Printer admin panels	admin	admin
SNMP	N/A	public (community string)

Other default weaknesses: default SNMP community strings (`public` = read, `private` = write), unnecessary services

(FTP, Telnet, plain HTTP, debug interfaces), default self-signed certs with known keys, and default DB accounts. Find them with Nmap service detection, **Hydra / Medusa** credential stuffing, Metasploit auxiliary modules, plain manual attempts (admin/admin), SNMP enumeration, and Shodan for internet-facing gear.

EXAM TELL Default credentials are a **top-priority, always-try-first** vector. Tools to name: **Hydra, Medusa, Metasploit auxiliary modules**. SNMP strings: **public (read), private (write)**. Default configs signal poor hygiene everywhere else, too.

1.5 running services

Every open port is attack surface that's active and accessible. Version detection lets you map each service to known CVEs.

PORT	SERVICE	ATTACK IMPLICATIONS
22	SSH	Brute force, weak keys, version vulns
23	**Telnet**	Cleartext credentials, MITM
25/465/587	SMTP	Open relay, user enumeration, phishing pivot
53	DNS	Zone transfer, cache poisoning, tunneling
80/443	HTTP/HTTPS	Web app attacks — XSS, SQLi, CSRF
135/139/445	**SMB**	EternalBlue, pass-the-hash, lateral movement
161/162	SNMP	Information disclosure, device control
389/636	LDAP	User enumeration, injection, pass-the-hash
1433	MSSQL	SQLi, xp_cmdshell, credential theft
3306	MySQL	SQLi, credential theft
3389	RDP	Brute force, BlueKeep, credential relay
5985/5986	WinRM	Remote command execution, lateral movement
8080/8443	HTTP Alt	Web app attacks, management interfaces

Scan types: `-sS` (stealth SYN), `-sV` (version detection), `-sU` (UDP — DNS, SNMP, TFTP), `-A` / NSE scripts (deep enumeration). Port states: **open** (listening — primary target), **closed** (reachable, nothing listening), **filtered** (firewall/ACL blocking), **open|filtered** (ambiguous, common with UDP).

EXAM TELL Know the port→service→attack mapping cold. Two that recur: **Telnet (23)** is always high priority because it sends credentials in **cleartext**; **SMB (445)** means EternalBlue and lateral movement. Match Nmap scan flags to purpose (`-sS -sV -sU -A`).

1.6 vulnerable encryption methods

Weak or misconfigured crypto lets an attacker decrypt captured traffic, run MITM, forge tokens/certs, or downgrade connections. Often overlooked, sometimes catastrophic.

PROTOCOL / ALGORITHM	STATUS	VULNERABILITY
SSL 2.0 / 3.0	Deprecated	POODLE, DROWN
TLS 1.0 / 1.1	Deprecated	BEAST, POODLE
TLS 1.2	Current (strong ciphers)	Safe with good config
TLS 1.3	Current / preferred	Most secure
RC4	Broken	Biased keystream, statistical attacks
DES / 3DES	Deprecated	SWEET32, insufficient key length
MD5	Broken (signatures)	Collision attacks
SHA-1	Deprecated (certs)	Collisions (SHAttered)
RSA-512 / 1024	Weak	Factoring feasible
WEP	Broken	IV reuse, statistical attacks
WPA (TKIP)	Deprecated	TKIP vulnerabilities

Weak cipher suites to flag: **NULL** (no encryption), **EXPORT** (FREAK), **RC4**, **DES/3DES** (SWEET32), and any without forward secrecy (no DHE/ECDHE). Named attacks: **POODLE** (SSL 3.0 downgrade), **BEAST** (CBC in TLS 1.0), **Heartbleed** (CVE-2014-0160, OpenSSL memory disclosure), **FREAK** (forces export ciphers). Cert issues: self-signed, expired, keys < 2048-bit, risky wildcards, SHA-1 signatures. Find it with `nmap --script ssl-enum-ciphers`, testssl.sh, SSLyze, Nikto, Wireshark, and scanners.

EXAM TELL Deprecated protocols: **SSL 2.0/3.0, TLS 1.0/1.1**. Broken algorithms: **RC4, DES, MD5, SHA-1**. Wireless: **WEP broken, WPA/TKIP deprecated**. The tool that enumerates weak cipher suites is Nmap's `ssl-enum-ciphers` script.

1.7 defensive capabilities

This isn't just about evading detection — it's about finding where the gaps are and how aggressively you can operate. The defensive landscape drives technique selection.

- **IDS/IPS** — signature-based (evade by obfuscating payloads) vs anomaly-based (evade by going slow); NIDS (evade via encryption/fragmentation/tunneling) vs HIDS (hard to evade once on-host).
- **SIEM** — aggregates and correlates logs; only detects what it's configured to see; log tampering becomes relevant.
- **EDR** — CrowdStrike, Carbon Black, SentinelOne detect behavioral anomalies; bypass with **LOLBins**, fileless attacks, process injection; payload obfuscation becomes critical.
- **WAF** — filters HTTP for SQLi/XSS/traversal; bypass with encoding, obfuscation, unusual HTTP methods, fragmentation; presence ≠ secure app.
- **Network segmentation** — VLANs/firewalls limit lateral movement; misconfigured ACLs and dual-homed hosts are high-priority gaps.
- **Logging & monitoring** — poor logging = easier to operate undetected; clearing Windows Event Logs may be needed for stealth.
- **MFA** — makes credential attacks harder; bypass with MFA-fatigue, session-token theft, real-time phishing (Evilginx); its absence makes credential attacks far more viable.

DEFENSIVE GAP	ATTACK OPPORTUNITY
No IDS/IPS	Aggressive scanning without stealth
No MFA	Password spraying, credential stuffing
No EDR	Traditional malware delivery
No segmentation	Unrestricted lateral movement
No log monitoring	Operate without concern for detection
No WAF	Direct web attacks without obfuscation

EXAM TELL Defensive capabilities **determine tool and technique selection**. Gaps (no MFA, no EDR, no segmentation) are attack opportunities. Two specifics they test: **LOLBins evade EDR**, and **WAF presence requires payload obfuscation** for web attacks.

PART 2 **Capability selection**

Matching tools, exploits, and techniques to prioritized targets — within scope, documented and defensible.

2.1 tool selection

Choosing the right tool for each phase, based on the target, the vulnerability, and engagement constraints.

principles

- **Match the tool to the target** — AD wants BloodHound/Impacket, web wants Burp/sqlmap, wireless wants Aircrack-ng.
- **Consider detection risk** — noisy (aggressive Nmap, default Metasploit payloads) vs stealthy (manual, LOLBins).
- **Understand limitations** — every tool has false positives/negatives; a web scanner won't find AD misconfigs.
- **Use purpose-built tools** — sqlmap beats a generic scanner for SQL injection.

CATEGORY	TOOLS	PRIMARY USE
Network scanning	Nmap, Masscan	Port/service discovery
Vulnerability scanning	Nessus, OpenVAS, Nikto	Vulnerability identification
Exploitation framework	Metasploit	Exploit delivery, post-exploitation
Web app testing	Burp Suite, ZAP, sqlmap	Web vulnerability exploitation
Password attacks	Hashcat, John, Hydra	Credential attacks
Active Directory	BloodHound, Impacket, CrackMapExec	AD enumeration & exploitation
Wireless	Aircrack-ng, Kismet	Wireless attacks
Packet crafting	Scapy, Impacket	Custom packet creation
Payload generation	msfvenom	Shellcode & payload creation
OSINT	theHarvester, Maltego, Recon-ng	Passive information gathering
Credential harvesting	Mimikatz, Responder	Credential extraction
Lateral movement	CrackMapExec, Impacket, Evil-WinRM	Post-exploitation movement
Cloud testing	Pacu, ScoutSuite, Prowler	Cloud environment testing

Also weigh engagement type (black vs white box), OS (Aircrack-ng is Linux, Mimikatz is Windows), detection risk (Metasploit's default signatures are known to most EDR), scope (traffic-heavy tools can breach RoE), and reliability (stable beats untested).

EXAM TELL Know the one-line purpose of each tool: **Metasploit = general exploitation, Burp = web apps, BloodHound = AD**, sqlmap = SQLi, Aircrack-ng = wireless, Mimikatz = credential dumping, msfvenom = payloads. Selection must also weigh **detection risk and scope limits**.

2.2 exploit selection & customization

Finding the exploit is half the job — it usually needs adapting to work reliably against the specific target.

selection process

- Identify the vulnerability — CVE ID, software version, OS.
- Search databases — Exploit-DB, Metasploit modules, GitHub, Packet Storm.
- Assess quality — some exploits are reliable, some unstable, some need specific conditions.
- Check compatibility — correct version and platform.
- Test in a lab — never deploy untested exploits against production.

Quality indicators to weigh: author reputation, date, stated test environment, dependencies, and stability notes (does it crash the service? can it run more than once?).

METASPLOIT RATING	MEANING
Excellent	No target service instability
Great	Default target, auto-detects version
Good	Default target, no auto-detection
Normal	Reliable but may require configuration
Average	Difficult, may not work reliably
Low	Very difficult to exploit
Manual	Requires significant manual interaction

Raw public exploits often need: shellcode replacement (msfvenom), target adjustment (memory addresses/offsets/return addresses per version), payload encoding (evade IDS/EDR), a changed delivery mechanism, an updated callback IP/port, and the right privilege context.

2.2.1 code analysis — read before you run

Reading exploit source before deploying it is mandatory for safety, customization, reliability assessment, and understanding the vulnerability. What to look for: hardcoded values (IPs, ports, offsets, shellcode), dependencies, error handling (poor handling crashes services), the delivery mechanism, cleanup, and the IOCs it leaves. By language: Python (hardcoded values, sockets, shellcode), Ruby/Metasploit (module structure, options, targets), C/C++ (overflow mechanics, shellcode, compilation), Bash (commands and their impact).

> **DANGER PATTERNS** Public exploit code can hide malice. Watch for **backdoors** (access beyond the stated purpose), **data destruction** (delete/encrypt — ransomware behavior), **lateral propagation** (spreads beyond the target), and **exfiltration** (sends data to external servers). Never run unanalyzed exploit code against production.

> **EXAM TELL** Know the Metasploit ratings order (**Excellent → Great → Good → Normal → Average → Low → Manual**), that **code analysis is required before deploying public exploits**, that exploits commonly need customization (shellcode, IPs, offsets), and that **msfvenom** is the payload-generation tool.

2.3 documentation

Documentation both guides execution and provides report evidence. Three artifacts, three audiences.

2.3.1 attack path

A documented sequence from initial access to objective, connecting individual vulnerabilities into one exploitation chain. Components: entry point → initial foothold → privilege escalation → lateral movement → persistence → objective.

```
[Internet] → phishing email → user clicks → foothold: WS01 user → priv-esc (unquoted service path) → SYSTEM on WS01 → Mimikatz dump → FS01 (captured creds) → HR share → DC01 (pass-the-hash) → Domain Admin → DCSync
```

Value: a roadmap for the tester, proof-of-compromise for the client, the critical-link view for remediation, and the basis of the report's Attack Narrative. Mapping tools: **BloodHound** (AD paths, automatic), MITRE ATT&CK Navigator (TTP mapping), manual diagramming (draw.io, Visio).

2.3.2 low-level diagram vs 2.3.3 storyboard

ASPECT	LOW-LEVEL DIAGRAM	STORYBOARD
Audience	Technical staff (engineers, admins)	Executives, management
Detail	High — IPs, ports, tools, commands	Low — business impact focus
Language	Technical	Business-friendly
Purpose	Reproduction & remediation	Decision-making & awareness
Format	Network diagrams, flowcharts	Slides, infographics, narrative

The low-level diagram carries network topology, hostnames/OS/ports, numbered steps with tools and commands, data flows, and privilege levels. The storyboard tells the story ("phishing → clicked → access → escalated → customer DB"), states business impact ("50,000 records, $2M fine"), names the few key vulnerabilities, and recommends actions — all in plain language.

EXAM TELL The recurring match-up: **low-level diagram = technical audience; storyboard = executive audience**. Attack paths connect vulnerabilities into chains, and **BloodHound generates attack paths for AD**. Documentation serves both execution and the final report.

2.4 dependencies

Prerequisites that must exist for an exploit or path to succeed. Mapping them prevents wasted effort and dead-end chains.

- **Technical** — software version (an Apache 2.4.49 exploit fails on 2.4.50), OS, architecture (32- vs 64-bit shellcode must match), service state, required libraries.
- **Access** — network reachability to the port, authentication to reach the vulnerable code, privilege level before exploitation.
- **Timing** — race conditions needing precise timing, active-session requirements, service availability at exploit time.
- **Chain** — later steps depend on earlier successes; if initial access fails, the whole chain fails.

Example — pass-the-hash to a DC depends on: (1) an NTLM hash of a domain admin (from prior dumping), (2) SMB 445 reachable, (3) SMB signing not enforced, (4) Impacket/CrackMapExec ready, (5) no blocking firewall ACLs. Dependency mapping prevents wasted attempts, mid-engagement surprises, dead-ends, and scope violations from out-of-scope prerequisites.

EXAM TELL Exploits carry **specific version, OS, and architecture requirements** (32- vs 64-bit is a favorite). Attack chains have dependencies — later steps need earlier ones. Network access, auth requirements, and service availability are all dependencies, and analyzing them up front prevents wasted effort.

2.5 consideration of scope limitations

The legal and ethical boundaries of the test. Operating outside scope isn't just an RoE breach — it can be a crime.

Scope lives in the **Statement of Work (SoW)** and **Rules of Engagement (RoE)**, specifying in/out-of-scope systems, permitted techniques, testing windows, geographic limits, and third-party authorization.

SCOPE LIMITATION	IMPLICATION FOR ATTACK PLANNING
"No DoS attacks"	No stress testing or flood attacks
"No social engineering"	No phishing or pretexting
"Production excluded"	Test staging/dev only
"No third-party systems"	Can't attack cloud infra directly
"Window: 2AM–6AM only"	Schedule scans and attacks accordingly
"No persistent backdoors"	Remove access mechanisms each session

Scope creep is discovering interesting systems or paths outside the defined scope — a vulnerable third-party API, a path through an out-of-scope box to an in-scope target, a flaw in a cloud service. Handle it: **stop and document**, do not exploit; **notify the client** and seek authorization; find alternative in-scope paths; note significant out-of-scope risk in the report. Cloud (AWS/Azure/GCP) and SaaS need explicit provider authorization. Violations invoke the **CFAA** (US) and **CMA** (UK), plus civil and criminal liability.

EXAM TELL Scope is defined in the **SoW and RoE**; violations carry legal consequences (**CFAA, CMA**); third-party/cloud needs separate authorization. When a path runs through an out-of-scope system, the answer is always **document, notify, seek authorization** — never proceed.

2.6 labeling sensitive systems

Some in-scope systems are so critical that aggressive testing could cause real harm. Labeling them ensures appropriate care.

Sensitivity comes from business criticality, data sensitivity (PII, financial, healthcare, IP), regulation (PCI DSS, HIPAA, SOX), or safety (ICS/SCADA, medical devices). Examples: production databases (live data can corrupt), payment systems (PCI DSS), medical devices (HIPAA, patient care), **ICS/SCADA** (physical damage/safety), core network gear (whole-network outage), Domain Controllers (lockouts, directory corruption), backup systems (blocks real recovery).

Handle them: identify pre-engagement during scoping; document each with handling requirements; define permitted testing types; prefer **replica/staging environments** over production; test in maintenance windows; keep a rollback plan. In diagrams, flag sensitive systems with a distinct color/icon/border, the sensitivity reason, constraints, and the owner's contact. Recon indicators: naming (`prod-db01`, `payment-server`, `ics-controller`), regulatory tags, separate VLANs/DMZs, client asset inventories.

EXAM TELL Sensitive systems can be **in-scope but require special handling**. **ICS/SCADA has safety implications**; PCI DSS / HIPAA / SOX create special testing requirements; and **replica/staging is preferred** for testing sensitive production systems.

SYNTHESIS

Correlating concepts

The high-value combinations — where two or three factors stack into an obvious answer.

`High CVSS+High EPSS+EOL=ultimate priority`

Severe, actively exploited, and never going to be patched. No combination produces a more urgent target.

EXAM APPLICATION An EOL Windows Server 2008 with CVE-2017-0144 (EternalBlue, CVSS 9.8) and a high EPSS score is your first attack target.

Default configs+Running services=quick win

A service on a standard port plus known default credentials is trivially exploitable. Try defaults before any complex exploit.

EXAM APPLICATION Nmap shows 8080 open, banner says Apache Tomcat → immediately try default manager creds (admin/admin, tomcat/tomcat).

Defensive capabilities+Tool selection=stealth vs speed

The defensive landscape sets your aggression. No IDS/IPS → `-T4/-T5`, run fast. IDS present → `-T1/-T2`, fragment, LOLBins. EDR → fileless/LOLBins, no dropped executables. WAF → encode payloads, unusual HTTP methods, test bypass.

EXAM APPLICATION "Which technique given this environment?" is answered by the defenses described in the scenario.

Scope limits+Sensitive systems=professional judgment

When a path runs through a sensitive or out-of-scope system: document, don't exploit without authorization, notify the client, find an in-scope alternative.

EXAM APPLICATION A path through an out-of-scope/sensitive system tests whether you stop-document-notify rather than proceed.

Attack path+Low-level diagram+Storyboard=complete documentation

Same information, three audiences: the logical sequence (planning/execution), the technical detail (engineers/remediation), the business narrative (executives/decisions).

EXAM APPLICATION "Which documentation type for [audience]?" requires knowing which format matches which reader.

EXAM How to attack a 4.1 scenario

A three-question filter that resolves most scenario items — plus what the questions actually test.

1

What is the target?

High-value? EOL? Default config? Sensitive? This sets base priority and required caution.

2

What are the constraints?

Scope limitations? Defensive capabilities? Dependencies? These disqualify otherwise-tempting answers.

3

What is the objective?

What does the engagement require you to achieve? The intersection of the three points to the answer.

what the questions test

- **Scenario prioritization** — given CVEs, CVSS, and system types, which first? Highest CVSS + highest EPSS + EOL +

network-accessible.

- **Metric interpretation** — read a CVSS vector; AV:N + PR:N + UI:N = highest exploitability.
- **Tool matching** — the right tool for a target type and objective.
- **Documentation audience** — low-level = technical, storyboard = executive.
- **Scope handling** — out-of-scope path → document, notify, seek authorization.
- **Sensitive-system handling** — replica environment, timing restrictions, special care.
- **Defensive response** — LOLBins for EDR, encoding for WAF, stealth timing for IDS.

REFERENCE

Keyword quick reference

Every must-know term with the one implication the exam attaches to it.

KEYWORD	CONTEXT	EXAM IMPLICATION
CVSS base score	Severity measurement	Higher score = higher priority (read the vector, not just the number)
EPSS	Exploitation probability	High EPSS = exploit code exists and is in use
CVE	Specific vulnerability ID (MITRE)	Used to find exploit code; format CVE-YEAR-NUMBER
CWE	Weakness category (MITRE)	Predicts similar vulnerabilities in the same app
EOL / EOS	Unsupported software	Permanent vulnerability, high priority
Default credentials	Out-of-box passwords	Quick win — always try first
Running services	Open ports	Define the attack surface
Weak encryption	SSL/TLS/cipher issues	Interception / downgrade opportunities
Defensive capabilities	IDS/IPS/EDR/WAF/MFA	Determine technique selection
Tool selection	Matching tool to target	Know which tool for which job
Exploit customization	Modifying public exploits	Shellcode, IPs, offsets
Code analysis	Reading exploit code	Safety and customization — do it first
Attack path	Exploitation sequence	Connects vulnerabilities to the objective
Low-level diagram	Technical documentation	For security engineers
Storyboard	Executive documentation	For non-technical stakeholders
Dependencies	Prerequisites for exploits	Version, access, architecture
Scope limitations	Legal boundaries	CFAA, CMA, SoW, RoE
Sensitive systems	Critical / regulated systems	Special handling required
High-value asset	Critical business system	Domain Controller, database, CA